
Speaker Identification via Partition-Based Search: A Comparative Study on VoxCeleb1

George Nazos (MTN2519), Konstantinos Kaimakis (MTN2508)

NCSR Demokritos UNIPi

June 2026

Abstract

For our assignment in Machine Learning in Multimodal data we created **speech-id**, a speaker identification system evaluated on VoxCeleb1 audio files that compares three vector search architectures across three speaker embedding models. Starting from a PostgreSQL + pgvector database of 144,840 enrollment utterances from 1,251 speakers, we evaluate: (i) exhaustive brute-force scan, (ii) metadata hierarchy routing (gender, nationality), and (iii) K -means cluster routing (256 clusters), each combined with three aggregation modes (speaker centroid, closest utterance, and vote). Experiments are conducted for ECAPA-TDNN (192 dim), WavLM X-Vector (512 dim), and WavLM mean-pooled (768 dim) embeddings across 8,676 test utterances. ECAPA-TDNN achieves near-perfect identification at 99.75% Top-1 under brute-force search, K -means using speaker's centroid reaches 93.56% at only 0.015 ms/query, a $2,520\times$ speed-up over brute-force closest. WavLM X-Vector yields 97.42% Top-1, while naive WavLM mean-pooling collapses to 26–30% Top-1, confirming that task-specific fine-tuning dominates the choice of search strategy.

Contents

1	Introduction	2
2	Related Work	2
3	Dataset and Split	2
4	Embedding Models	3
4.1	ECAPA-TDNN (192-dim)	3
4.2	WavLM X-Vector (512-dim)	3
4.3	WavLM Mean-Pooled (768-dim)	3
5	Database Architecture	4
5.1	Schema Design	4
5.2	Population Pipeline	5
6	Search Architectures	5
6.1	Brute-Force	6
6.2	Metadata Hierarchy	6
6.3	K-Means Cluster Routing	7
6.4	Aggregation Modes	7
7	Evaluation Metrics	7
8	Experimental Setup	8
9	Results	8
9.1	ECAPA-TDNN (192-dim)	8
9.2	WavLM X-Vector (512-dim)	9
9.3	WavLM Mean-Pooled (768-dim)	9
9.4	Metadata Hierarchy: Varying Probe Depth	9
9.5	Cross-Model Comparison	11
10	Analysis	11
10.1	Effect of Search Architecture	11
10.2	Effect of Aggregation Mode	12
10.3	Embedding Model Comparison	13
11	Conclusions	13
A	Database Schema SQL	14

1 Introduction

Speaker identification, determining which enrolled speaker produced a given utterance, is a core component of multi-speaker meeting transcription, voice-based access control, and personalised voice assistants. In a closed system, identifying a speaker reduces to a nearest-neighbour search against the enrolled voice data. At database sizes of tens of thousands of utterances, the naive exhaustive search becomes a bottleneck.

This report documents an evaluation of how three search architectures interact with three embedding models of varying dimensionality and training objective. All experiments share a common PostgreSQL + pgvector backend and are run on 8,676 unseen test samples from VoxCeleb1.

Our findings are:

1. A reproducible evaluation framework built on PostgreSQL + pgvector with separate schemas per embedding model.
2. A three-way comparison of search strategies: brute-force, metadata-gated, and unsupervised cluster routing.
3. An analysis of three aggregation modes (centroid, closest, vote) across all architecture-model combinations.

2 Related Work

Speaker embeddings. x -vectors [1] are fixed-length speaker embeddings extracted by a Time Delay Neural Network (TDNN), a feed-forward architecture that models temporal context by applying convolutions over dilated time steps. In the x -vector method, a statistics pooling layer aggregates the frame-level TDNN outputs into a single utterance-level vector (mean and standard deviation), which is then projected to a compact speaker representation. ECAPA-TDNN extends this by adding multi-scale Res2Net aggregation and channel-dependent frame attention [2], achieving state-of-the-art performance on VoxCeleb. WavLM [3] applies masked speech prediction as a self-supervised pre-training objective over 94,000 hours of audio, producing rich 768-dimensional representations that transfer well to downstream tasks when fine-tuned.

3 Dataset and Split

VoxCeleb1 [4] contains audio from 1,251 celebrities (690 male, 561 female) extracted from YouTube interviews, sampled at 16 kHz, stored as .wav files.

Train/test split. We adopt a *leave-one-video-out* strategy: `make_test_split.py` moves the lexicographically last video directory of each speaker to the test set, minimizing the chance that test utterances come from a recording session unseen during enrollment.

Table 1: Dataset statistics after leave-one-video-out split.

Partition	Utterances	Speakers
Enrollment (train)	144,840	1,251
Test	8,676	1,251
Total	153,516	1,251

4 Embedding Models

Three embedding extractors are evaluated, each stored in an isolated PostgreSQL schema.

4.1 ECAPA-TDNN (192-dim)

[speechbrain/spkrec-ecapa-voxceleb](#) this system is composed of an ECAPA-TDNN model. It is a combination of convolutional and residual blocks [2, 5]. The model encodes a 16 kHz waveform into a **192-dimensional** L2-normalised embedding. The model was already fine-tuned on VoxCeleb2 for speaker verification using additive angular margin loss.

4.2 WavLM X-Vector (512-dim)

[wavlm-base-plus-for-speaker-verification](#) is a WavLM-Base+ transformer backbone is combined with an *x*-vector head (statistics pooling and linear projection) fine-tuned on VoxCeleb1 for speaker verification, producing **512-dimensional** L2-normalised embeddings. Because this model has been exposed to speaker verification objectives during fine-tuning, it is directly comparable to ECAPA-TDNN in terms of training signal.

4.3 WavLM Mean-Pooled (768-dim)

[microsoft/wavlm-base-plus](#) [3] is used *without any task-specific fine-tuning*: hidden states from all 12 transformer layers are mean-pooled into a **768-dimensional** vector, clipped to ≤ 20 s due to VRAM constraints. This configuration tests whether generic representations carry sufficient speaker-discriminative information for closed-set identification without additional adaptation.

5 Database Architecture

5.1 Schema Design

PostgreSQL 16 with pgvector 0.4.2 is deployed via Docker. Each embedding model occupies a separate schema (public for ECAPA, wavlm_xvector, wavlm) with identical table structure but with specific vector dimension for each model.

Table 2: Database tables per schema. {dim} is model-specific (192/512/768).

Table	Key	Rows	Purpose
speakers	speaker_id	1,251	Gender and nationality metadata
audio_embeddings	id (identity)	144,840	Per-utterance {dim}-dim embedding
metadata_centroids	(category, value)	58	Mean embeddings by gender/nationality
cluster_centroids	cluster_id	256	K -means cluster centres

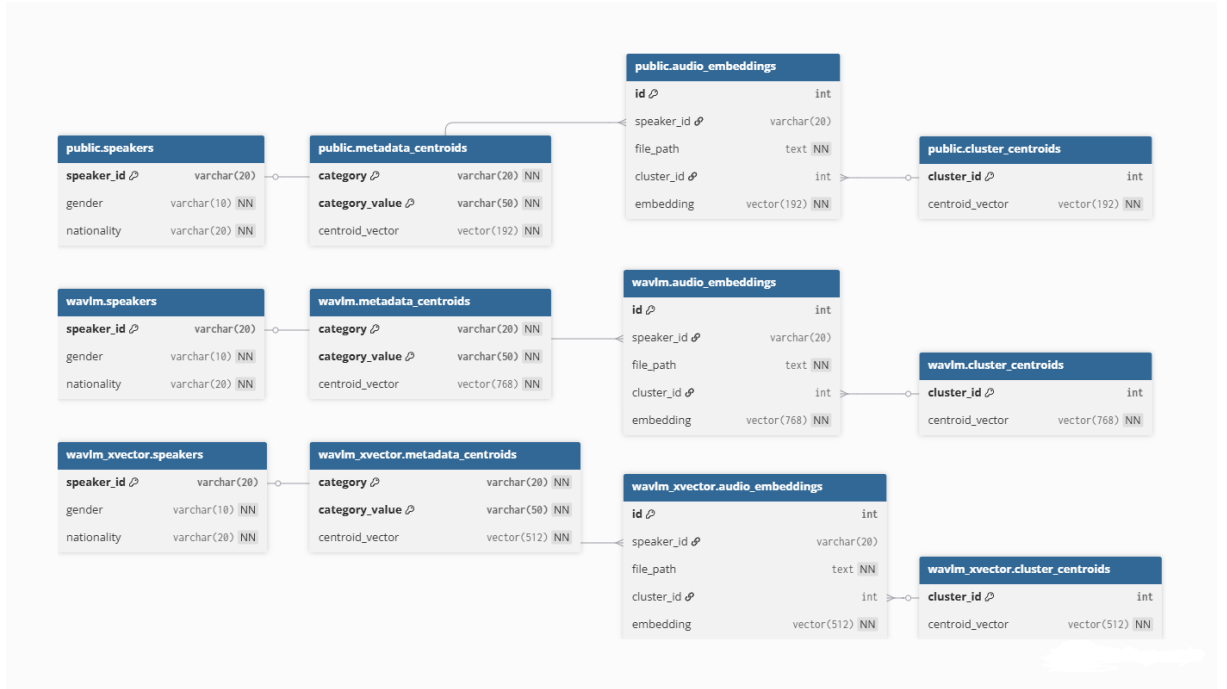


Figure 1: Entity-relationship diagram of the database schema. Each embedding model uses an identical structure in its own PostgreSQL schema, {dim} is substituted with 192, 512, or 768 at schema creation time.

The speakers table is the core of the schema: it stores one row per enrolled speaker with their gender and nationality, and every audio embedding row carries a foreign key back to it. The audio_embeddings table is the largest, holding one dimensional vector per utterance together

with a `cluster_id` that records which K -means partition the embedding was assigned to during the population phase. The `metadata_centroids` table stores the mean embedding for each (category, value) pair, two gender groups and 56 gender $\times$ nationality groups, so that the metadata routing step can identify the closest partition. Finally, `cluster_centroids` stores the 256 K -means centres used by the cluster-routing architecture, at query time the nearest cluster centroid is found and its `cluster_id` is passed as a `WHERE` filter to restrict the database scan to roughly $144,840/256 \approx 566$ candidate utterances.

5.2 Population Pipeline

`populate_db.py` runs in three phases:

1. **Ingestion:** loads the `.pt` embeddings file and VoxCeleb metadata CSV, inserts speakers, streams embeddings via PostgreSQL binary COPY, computes 58 metadata centroids (2 gender + 56 gender $\times$ nationality groups).
2. **Clustering:** fetches all embeddings, L2-normalises them, and runs `MiniBatchKMeans(K=256)`.
3. **Assignment:** inserts cluster centroids, updates every row in `audio_embeddings` with its `cluster_id`.

6 Search Architectures

Let $\mathbf{q} \in \mathbb{R}^d$ be the L2-normalised query embedding. All searches use cosine similarity $\text{sim}(\mathbf{a}, \mathbf{b}) = \mathbf{a}^\top \mathbf{b}$, implemented via the `<=>` operator.

Before every evaluation run, indexed search is disabled so that PostgreSQL performs exact sequential scans regardless of architecture:

```
SET enable_indexscan = off;
SET enable_indexonlyscan = off;
SET enable_bitmapscan = off;
```

Listing 1: Index scan disabled before every evaluation run.

6.1 Brute-Force

```
SELECT ae.id, ae.speaker_id,
       ae.embedding <=> %s::vector AS distance
FROM   audio_embeddings ae
JOIN   speakers s ON ae.speaker_id = s.speaker_id
ORDER BY ae.embedding <=> %s::vector ASC
LIMIT %s;
```

Listing 2: Brute-force utterance-level query.

6.2 Metadata Hierarchy

Two in-memory comparisons route the query before issuing a filtered SQL query:

1. $g^* = \arg \max_g \langle \mathbf{q}, \boldsymbol{\mu}_g \rangle$ over the 2 gender centroids.
2. Top- P nationalities from gender $\times$ nationality centroids matching g^* .
3. Filtered DB query:

```
SELECT ae.id, ae.speaker_id,
       ae.embedding <=> %s::vector AS distance
FROM   audio_embeddings ae
JOIN   speakers s ON ae.speaker_id = s.speaker_id
WHERE  s.gender = %s
      AND s.nationality = ANY(%s)
ORDER BY ae.embedding <=> %s::vector ASC
LIMIT %s;
```

Listing 3: Metadata-hierarchy filtered query.

6.3 K-Means Cluster Routing

Top- C nearest cluster centroids ($C=1$) are identified then only utterances in those clusters are searched:

```
SELECT ae.id, ae.speaker_id,
       ae.embedding <=> %s::vector AS distance
FROM   audio_embeddings ae
JOIN   speakers s ON ae.speaker_id = s.speaker_id
WHERE  ae.cluster_id = ANY(%s)
ORDER BY ae.embedding <=> %s::vector ASC
LIMIT %s;
```

Listing 4: K -means cluster-routed query.

With $K=256$ and 144,840 utterances, each cluster holds on average ≈ 600 embeddings.

6.4 Aggregation Modes

- **Speaker centroid:** Per-speaker mean embeddings are loaded from the database once at startup and held in memory. Per-query ranking is a pure in-memory cosine dot-product:
- **Closest utterance:** The speaker of the single nearest DB result is returned.
- **Vote:** The top-25 DB results are numbered by `speaker_id`, the speaker with the most votes wins, ties broken by best distance.

7 Evaluation Metrics

- **Top-1%:** fraction of test queries for which the correct speaker is ranked first, expressed as a percentage.
- **Hit@10%:** fraction of queries for which the correct speaker appears anywhere in the top-10 results.
- **Macro-F1%:** harmonic mean of per-speaker precision and recall, averaged equally over all 1,251 speakers.
- **Latency (ms/query):** mean time per query measured with `time.perf_counter_ns` over all 8,676 test utterances reported in milliseconds.

All classification metrics are macro-averaged over 1,251 speakers. Latency is measured with `time.perf_counter_ns` and reported as mean milliseconds per query over 8,676 test utterances.

8 Experimental Setup

Table 3: Hyperparameters shared across all model experiments.

Parameter	Default Value
Distance metric	Cosine ($\leq$ or $\geq$)
Clusters K	256
Cluster probe depth C	1
Metadata probe depth P	1
Top- K for Hit@ K	10
Candidate limit (vote)	25
Test queries	8,676
Utterance mode	all

9 Results

9.1 ECAPA-TDNN (192-dim)

Table 4: ECAPA-TDNN results (cosine, $C=1$, schema: public). **Bold**: best per column overall, underline: best non-exact (non-brute-force) configuration.

Architecture	Aggregation	Top-1%	Hit@10%	F1%	ms
Brute-force	Speaker centroid	99.64	99.97	98.99	0.030
	Closest	99.75	99.94	99.06	37.81
	Vote	99.68	99.94	98.97	37.86
Metadata hier.	Speaker centroid	33.24	33.29	28.94	0.032
	Closest	33.29	33.30	29.21	30.47
	Vote	33.25	33.30	29.12	30.21
K -Means	Speaker centroid	<u>93.56</u>	<u>93.75</u>	<u>88.81</u>	0.015
	Closest	<u>93.29</u>	<u>93.66</u>	<u>88.52</u>	48.25
	Vote	85.10	<u>93.66</u>	77.17	48.19

9.2 WavLM X-Vector (512-dim)

Table 5: WavLM X-Vector results (cosine, $C=1$, schema: wavlm_xvector).

Architecture	Aggregation	Top-1%	Hit@10%	F1%	ms
Brute-force	Speaker centroid	95.84	99.10	92.55	0.051
	Closest	97.42	99.65	94.96	238.34
	Vote	96.83	99.65	94.03	237.26
Metadata hier.	Speaker centroid	32.83	33.62	28.50	0.038
	Closest	33.07	33.63	28.59	26.24
	Vote	33.10	<u>33.63</u>	28.74	26.47
K -Means	Speaker centroid	<u>93.44</u>	<u>95.78</u>	<u>88.94</u>	0.023
	Closest	<u>93.04</u>	<u>95.95</u>	<u>88.18</u>	14.01
	Vote	81.13	<u>95.95</u>	73.12	14.04

9.3 WavLM Mean-Pooled (768-dim)

Table 6: WavLM mean-pooled results (cosine, $C=1$, schema: wavlm).

Architecture	Aggregation	Top-1%	Hit@10%	F1%	ms
Brute-force	Speaker centroid	26.19	49.87	18.01	0.039
	Closest	26.96	51.90	19.01	259.54
	Vote	29.66	53.08	20.44	258.10
Metadata hier.	Speaker centroid	7.46	12.79	4.99	0.032
	Closest	7.04	12.49	5.20	21.14
	Vote	7.35	12.66	4.89	21.13
K -Means	Speaker centroid	<u>23.62</u>	<u>43.11</u>	<u>16.39</u>	0.081
	Closest	17.95	34.64	12.72	14.45
	Vote	14.20	35.18	9.02	14.43

9.4 Metadata Hierarchy: Varying Probe Depth

The following tables extend the metadata hierarchy results by varying the nationality probe depth $P \in \{2, 5, 10\}$. $P=1$ results appear in the per-model tables above.

Table 7: ECAPA-TDNN — metadata hierarchy with varying probe depth P .

P	Aggregation	Top-1%	Hit@10%	F1%	ms
	Speaker centroid	47.71	47.78	41.31	0.044
2	Closest	47.75	47.79	41.84	32.79
	Vote	47.73	47.79	41.85	33.00
	Speaker centroid	65.86	65.99	56.45	0.072
5	Closest	65.91	66.00	57.39	33.36
	Vote	65.89	66.00	57.43	33.52
	Speaker centroid	73.87	74.08	64.82	0.097
10	Closest	73.94	74.09	65.91	30.49
	Vote	73.93	74.09	65.86	30.54

Table 8: WavLM X-Vector — metadata hierarchy with varying probe depth P .

P	Aggregation	Top-1%	Hit@10%	F1%	ms
	Speaker centroid	57.38	58.96	50.44	0.058
2	Closest	57.92	58.97	50.85	45.75
	Vote	57.77	58.97	50.76	45.55
	Speaker centroid	80.97	83.36	74.10	0.082
5	Closest	81.79	83.49	75.29	67.58
	Vote	81.55	83.49	75.06	67.45
	Speaker centroid	89.53	92.30	83.44	0.110
10	Closest	90.56	92.47	84.77	82.46
	Vote	90.24	92.47	84.47	82.34

Table 9: WavLM Mean-Pooled — metadata hierarchy with varying probe depth P .

P	Aggregation	Top-1%	Hit@10%	F1%	ms
2	Speaker centroid	12.29	22.71	8.88	0.058
	Closest	12.13	22.71	9.27	35.80
	Vote	12.75	23.11	9.09	35.69
5	Speaker centroid	21.35	40.69	15.04	0.159
	Closest	21.09	41.04	15.05	73.75
	Vote	22.76	41.94	15.86	74.05
10	Speaker centroid	24.05	46.62	16.61	0.285
	Closest	24.29	47.36	17.17	114.96
	Vote	26.34	48.37	17.98	116.52

9.5 Cross-Model Comparison

Table 10: Cross-model comparison at selected operating points.

Configuration	Metric	ECAPA	WavLM-XV	WavLM
BF + closest	Top-1%	99.75	97.42	26.96
	ms/query	37.81	238.34	259.54
BF + centroid	Top-1%	99.64	95.84	26.19
	ms/query	0.030	0.051	0.039
K-Means + centroid	Top-1%	93.56	93.44	23.62
	ms/query	0.015	0.023	0.081
K-Means + closest	Top-1%	93.29	93.04	17.95
	ms/query	48.25	14.01	14.45

10 Analysis

10.1 Effect of Search Architecture

Brute-force. For ECAPA-TDNN, brute-force delivers near-perfect Top-1 accuracy (99.75% closest, 99.64% centroid). For WavLM X-Vector the gap is larger (97.42% vs. 95.84%), indicating slightly less intra-speaker consistency in the 512-dim space. For WavLM mean-pooled, brute-force itself cannot exceed 29.66%, confirming that embedding quality is the binding constraint, no search strategy can recover from poorly representations.

Metadata hierarchy. Results collapse to $\approx 33\%$ for ECAPA and WavLM X-Vector and to $\approx 7\%$ for WavLM mean-pooled. We assume that the root cause is imbalance between nationalities, some may cover hundreds of speakers while others covers only a handful, creating highly imbalanced candidate pools. A routing error in the nationality selection step is unrecoverable because the filtered candidate pool then excludes the true speaker.

Effect of metadata probe depth. Tables 7–9 show that increasing the nationality probe depth P steadily recovers accuracy for all models. For ECAPA-TDNN, Top-1 rises from 33.3% at $P=1$ to 47.8%, 65.9%, and 73.9% at $P=2, 5$, and 10 respectively, confirming that the loss at $P=1$ is purely a routing coverage problem. For WavLM X-Vector the gains are larger. Top-1 jumps from 33.1% to 57.9% ($P=2$), 81.8% ($P=5$), and 90.6% ($P=10$), approaching brute-force closest (97.4%) at $P=10$ with the closest aggregation. For WavLM mean-pooled, even at $P=10$ Top-1 reaches only 26.3% (vote), barely above the brute-force baseline of 29.7%, confirming that the bottleneck is the embedding quality rather than the routing. In all cases, the speaker-centroid aggregation preserves near-zero latency (<0.3 ms) regardless of P , while closest and vote aggregation scale linearly with the enlarged candidate pool. However we must mention that in large scale scenarios the loading of centroids in memory at run time wouldn't be necessarily possible.

K-means cluster routing. For ECAPA, K-means centroid reaches 93.56% Top-1. The accuracy loss is attributable to routing errors, speakers who straddle cluster boundaries have their utterances split across clusters, and a single-probe search ($C=1$) misses any utterance not in the queried cluster. For WavLM X-Vector, K-means centroid achieves 93.44% at 0.023 ms, nearly matching ECAPA despite the larger embedding dimension. For WavLM mean-pooled, K-means degrades below brute-force (23.62% vs. 26.19%), indicating that the cluster structure in this space does not correlate with speaker identity.

10.2 Effect of Aggregation Mode

Speaker centroid vs. closest. Under brute-force, *closest* and *centroid* differ by at most 0.12 % (ECAPA) and 1.58 % (WavLM X-Vector) in Top-1. The cosine dot-product of the query against 1,251 pre-computed per-speaker mean vectors runs entirely in Python/NumPy, making centroid queries roughly $1,260\times$ faster than closest in the brute-force case (0.030 ms vs. 37.81 ms).

Vote aggregation. Aggregating the top-25 candidate utterances by majority vote degrades accuracy for K-means in all three models. With $C=1$, the candidate pool is dominated by speakers with many utterances in the winning cluster. Vote aggregation amplifies this positional bias, producing results consistently worse than simply selecting the closest utterance.

10.3 Embedding Model Comparison

The results reveal a clear three-tier hierarchy:

1. **ECAPA-TDNN**, best on all accuracy metrics, lowest latency for centroid search due to the smallest embedding dimension (192). Its task-specific fine-tuning produces well-calibrated cosine similarity scores.
2. **WavLM X-Vector**, 97.42% Top-1 under brute-force closest. The higher latency for closest search (238 ms vs. 37.81 ms for ECAPA) reflects the larger 512-dim vectors.
3. **WavLM mean-pooled**, completely unsuitable for closed-set speaker identification without fine-tuning. Best Top-1 is 29.66% (vote, brute-force), and K-means routing actively hurts performance relative to brute-force.

11 Conclusions

We evaluated 27 experimental configurations (3 models $\times$ 3 search architectures $\times$ 3 aggregation modes) on VoxCeleb1 with 8,676 test utterances against 1,251 enrolled speakers. The principal findings are:

1. **The use of task specific model embeddings is important.**
2. **Brute-force centroid is the practical optimum for ECAPA for a database of this scale.**
3. **K-Means centroid is the scalable real-time option.**
4. **Metadata hierarchy is not viable as a hard filter.**
5. **Vote aggregation degrades under K-Means routing due to dominant speaker.**

Future work should explore multi-probe K-Means ($C > 1$) to close the accuracy gap with brute-force, data base indexing and AAM-softmax fine-tuning of WavLM to evaluate whether accuracy can be improved.

A Database Schema SQL

```

1 CREATE SCHEMA IF NOT EXISTS {schema},
2 SET search_path TO {schema},
3
4 CREATE TABLE speakers (
5     speaker_id VARCHAR(20) PRIMARY KEY,
6     gender      VARCHAR(10) NOT NULL,
7     nationality VARCHAR(20) NOT NULL
8 ),
9 CREATE INDEX idx_spk_gn ON speakers(gender, nationality),
10
11 CREATE TABLE audio_embeddings (
12     id          INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
13     speaker_id  VARCHAR(20) REFERENCES speakers(speaker_id),
14     file_path   TEXT NOT NULL,
15     cluster_id  INT,
16     embedding   VECTOR({dim}) NOT NULL
17 ),
18 CREATE INDEX idx_ae_spk ON audio_embeddings(speaker_id),
19 CREATE INDEX idx_ae_cl  ON audio_embeddings(cluster_id),
20 CREATE INDEX idx_ae_hnsw ON audio_embeddings
21     USING hnsw(embedding vector_cosine_ops)
22     WITH (m=16, ef_construction=64),
23
24 CREATE TABLE metadata_centroids (
25     category      VARCHAR(20) NOT NULL,
26     category_value VARCHAR(50) NOT NULL,
27     centroid_vector VECTOR({dim}) NOT NULL,
28     PRIMARY KEY (category, category_value)
29 ),
30
31 CREATE TABLE cluster_centroids (
32     cluster_id      INT PRIMARY KEY,
33     centroid_vector VECTOR({dim}) NOT NULL
34 ),
35 ALTER TABLE audio_embeddings ADD CONSTRAINT fk_cl
36     FOREIGN KEY (cluster_id)
37     REFERENCES cluster_centroids(cluster_id),

```

Listing 5: Schema template (data_base/schema.template.sql).

References

- [1] D. Snyder, D. Garcia-Romero, G. Sell, D. Povey, and S. Khudanpur, “X-vectors: Robust DNN embeddings for speaker recognition,” in *Proc. IEEE ICASSP*, 2018, pp. 5329–5333.
- [2] B. Desplanques, J. Thienpondt, and K. Demuynck, “ECAPA-TDNN: Emphasized channel attention, propagation and aggregation in TDNN-based speaker verification,” in *Proc. INTERSPEECH*, 2020, pp. 3830–3834.
- [3] S. Chen *et al.*, “WavLM: Large-scale self-supervised pre-training for full stack speech processing,” *IEEE J. Sel. Topics Signal Process.*, vol. 16, no. 6, pp. 1505–1518, 2022.
- [4] A. Nagrani, J. S. Chung, and A. Zisserman, “VoxCeleb: A large-scale speaker identification dataset,” in *Proc. INTERSPEECH*, 2017, pp. 2616–2620.
- [5] M. Ravanelli *et al.*, “SpeechBrain: A general-purpose speech toolkit,” *arXiv:2106.04624*, 2021.
- [6] A. Kane, “pgvector: Open-source vector similarity search for PostgreSQL,” <https://github.com/pgvector/pgvector>, 2023.